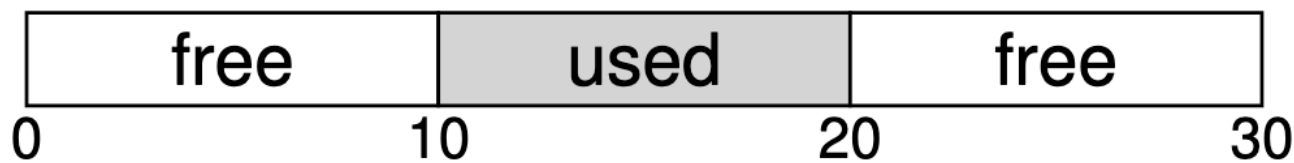


This chapter talks about fundamental aspects of any memory management system (malloc library or OS)

The free space gets chopped into little pieces of different sizes and is thus fragmented; subsequent requests may fail because there is no single contiguous space that can satisfy the request, even though the total amount of free space exceeds the size of the request.

Example,



Here, there are two fragmented chunks of free memory. A request for 15 bytes will even though there are 20 bytes free.

CRUX: HOW TO MANAGE FREE SPACE

How should free space be managed, when satisfying variable-sized requests? What strategies can be used to minimize fragmentation? What are the time and space overheads of alternate approaches?

Assumptions:

↳ We assume a basic interface such as that provided by `malloc()` & `free()`.

As `free()` doesn't take size, the library has to figure out its size given a pointer to memory chunk.

This library manages space called heap & uses a free list, which contains references to all the free chunks in space in the managed region of memory.

↳ Once a memory is allocated, it cannot be relocated to another location in memory.

Why?

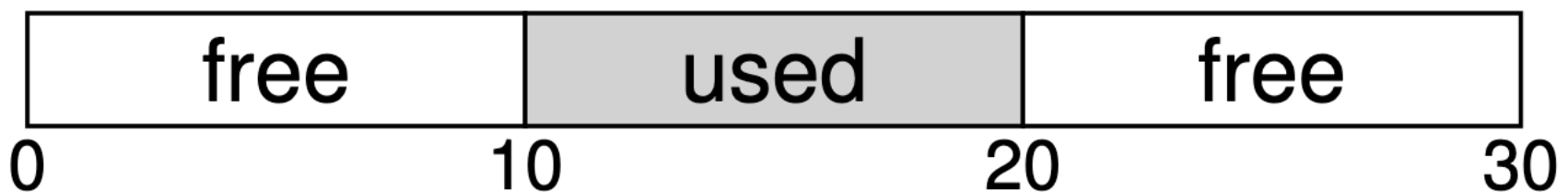
Once you hand a pointer to a chunk to a C program, it is difficult to determine all references (pointers) to that region, which may be stored in other variables or even registers at a given point in execution.

So, compaction is NOT possible.

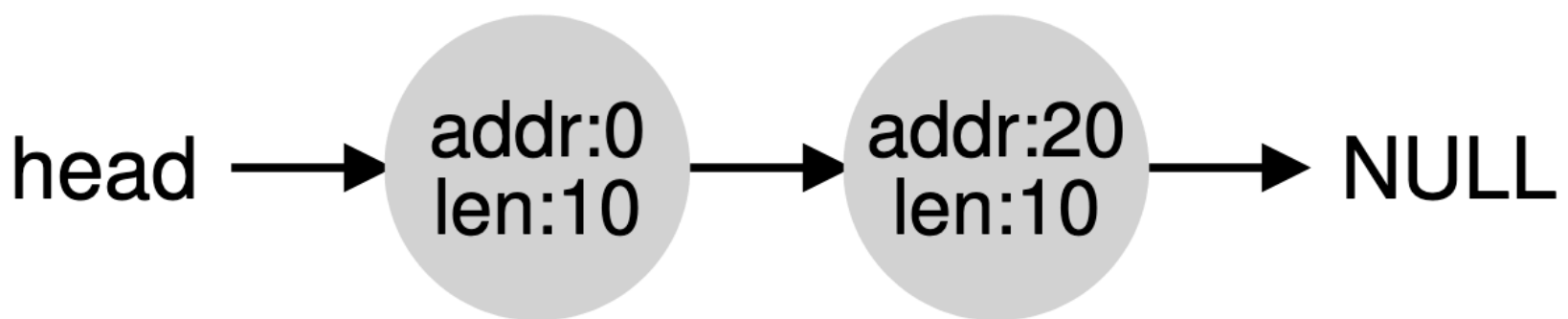
↳ Allocator manages a continuous region of bytes, which is of fixed size.

Low-Level Mechanisms

A free list contains a set of elements that describe the free space in heap.



A free list for the above 30-byte heap is



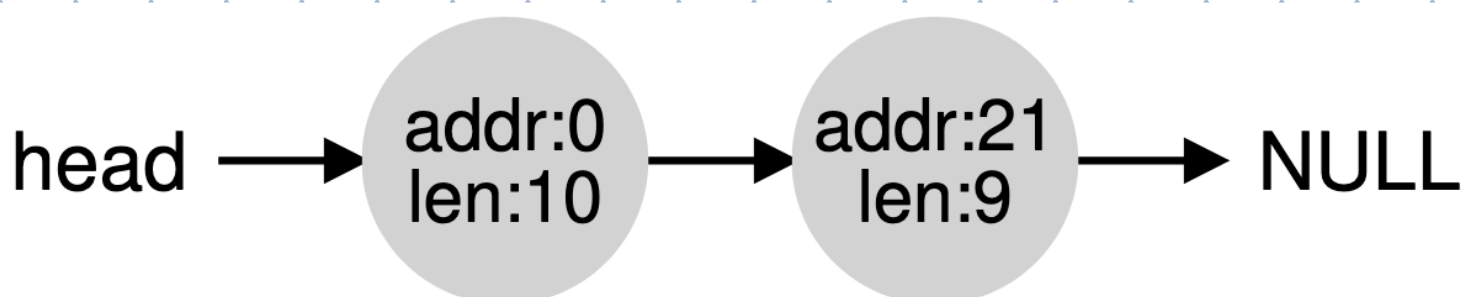
• Splitting

↳ Assume we get a request for a single byte of memory.

In this, the allocator will perform splitting

↳ Find a free chunk of space that can satisfy the request & split it into two.

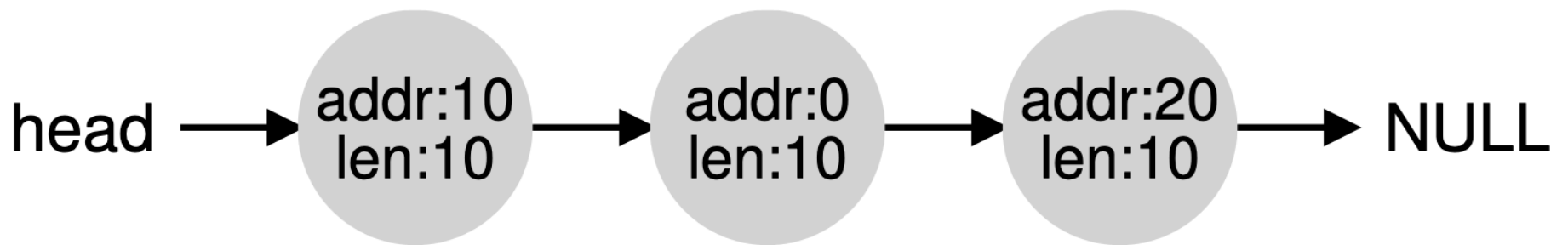
Thus, our free list would look like



↳ The allocator chose 2nd chunk. No reason is given as to why not 1st.

• Coalescing

In above example, if the allocated 10 bytes is freed, then our free list is:



Now, our entire heap is free but a request of 20 bytes will be rejected as free space is in chunks.

So, allocators perform coalescing:

↳ look at addresses of newly freed space & nearby chunks. If they are next to each other, merge them.

Our list will look as:



• Tracking the size of allocated regions:

↳ Most allocators store a little bit of extra information in a header block just before the handed out chunk of memory.

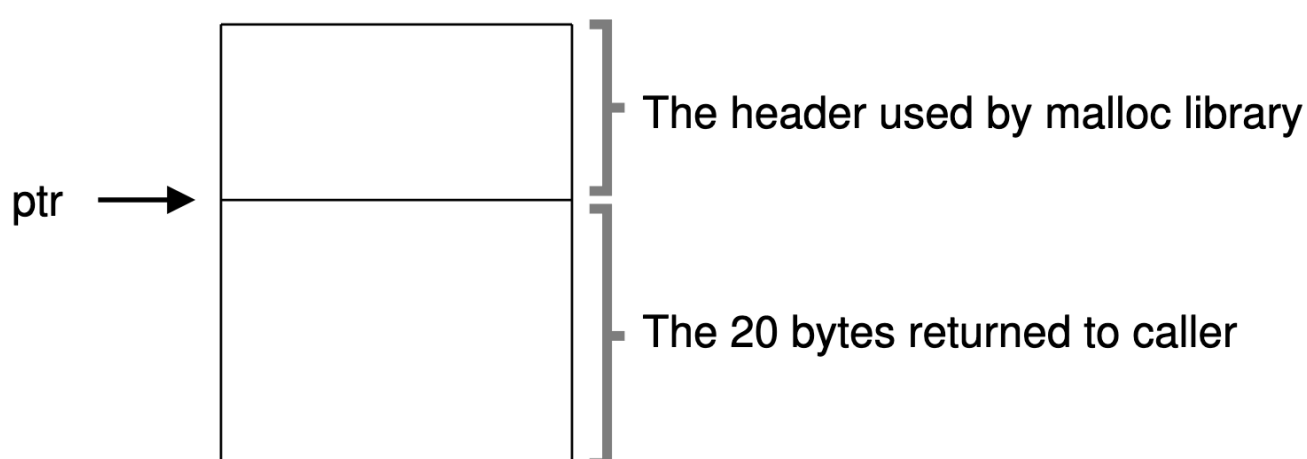


Figure 17.1: An Allocated Region Plus Header

6 The header contains the size of the allocated region
A magic number.

It may also have an additional pointer to speed-up deallocation.

```
typedef struct {  
    int size;  
    int magic;  
} header_t;
```

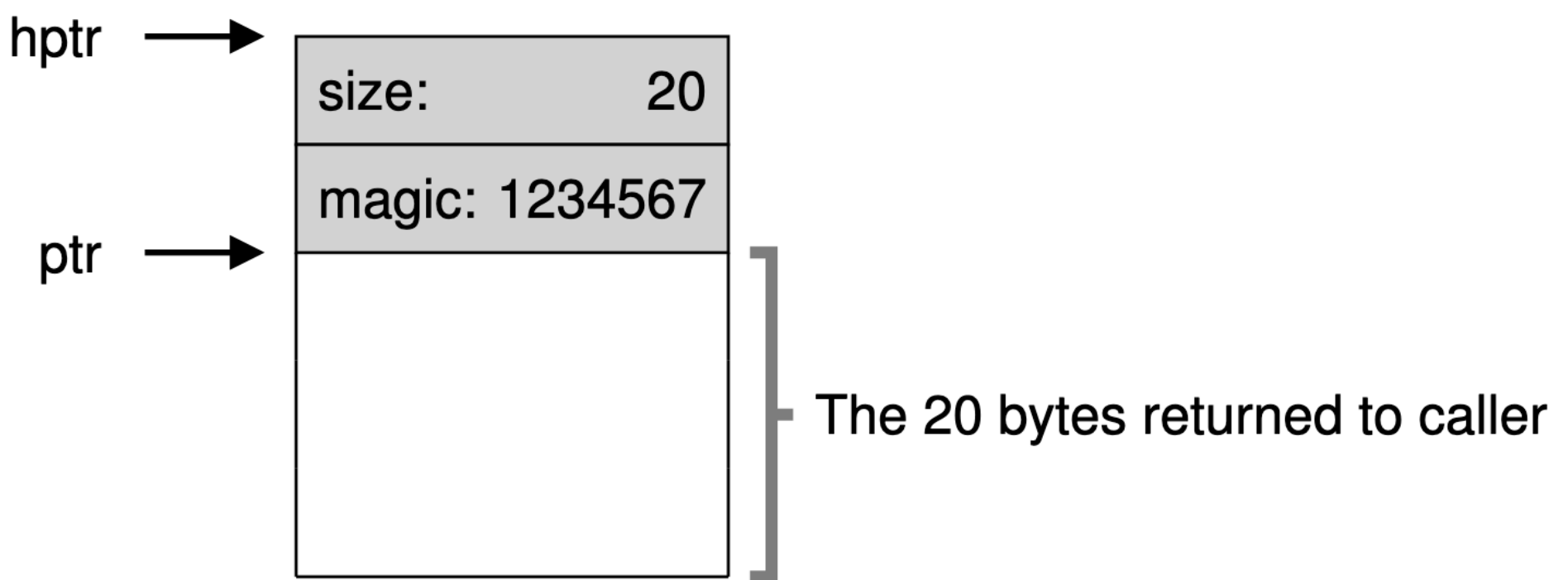


Figure 17.2: Specific Contents Of The Header

In fig 17.2, when user calls free, the library performs pointer arithmetic to find where the header begins:

```
void free(void *ptr) {  
    header_t *hptr = (header_t *) ptr - 1;  
    ...  
}
```

Now, as a sanity, it checks if the magic number is equal to expected value

```
(assert(hptr->magic == 1234567))
```

and calculate size to be freed

$hptr \rightarrow size + \text{size of header}$

• Embedding a free list

§ For a typical list, you call `malloc()` to allocate space for a node.

For a memory allocation library, you can't do this. Instead, it builds the free list inside the free space itself.

§ Assume we have a 4096 (4KB) chunk of memory to manage.

We define the node as:

```
typedef struct __node_t {  
    int size;  
    struct __node_t *next;  
} node_t;
```

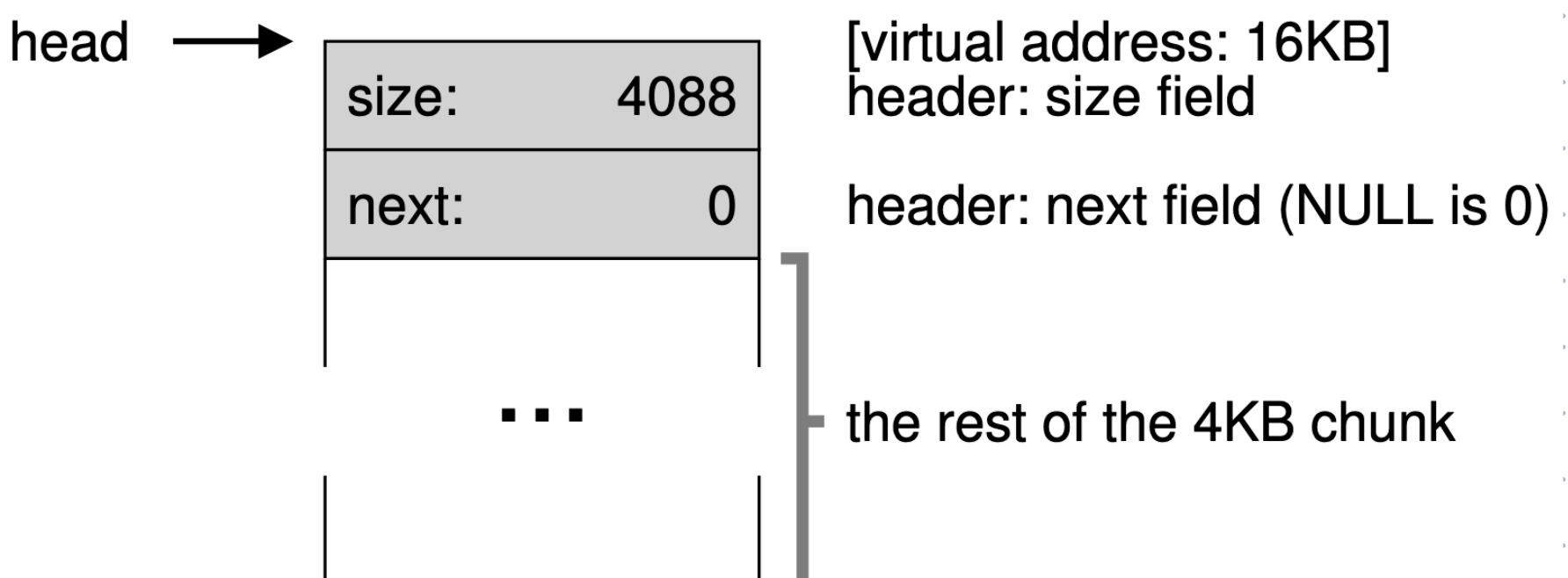
We build the heap as:

```
// mmap() returns a pointer to a chunk of free space  
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,  
                    MAP_ANON|MAP_PRIVATE, -1, 0);  
head->size = 4096 - sizeof(node_t);  
head->next = NULL;
```

space that is available to use.

→ The `head` ptr has start address of heap i.e. 16384

At first, it will look as:



Now, a chunk of 100 bytes of memory is requested. The library will find chunk that is large enough to accommodate the request.

As there is only one chunk, it is split into two:

- One chunk to serve the request (100 bytes) + header (8 bytes) = 108 bytes total

- The remaining free chunk.

Thus, the library allocated 108 bytes & returned a pointer to it.

The heap looks like

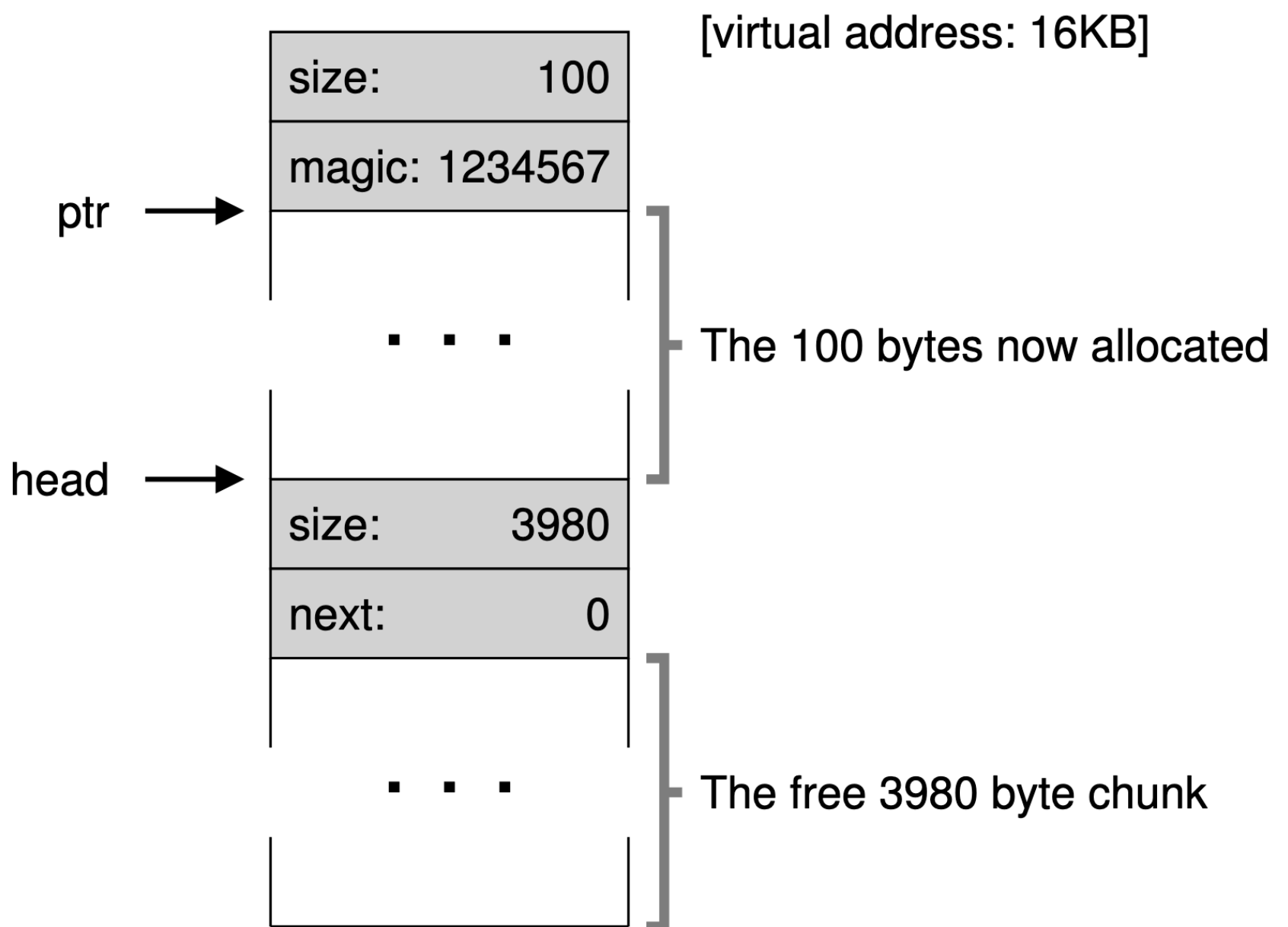


Figure 17.4: A Heap: After One Allocation

If there are 3 allocated region each of 108 byte, the heap will look like.

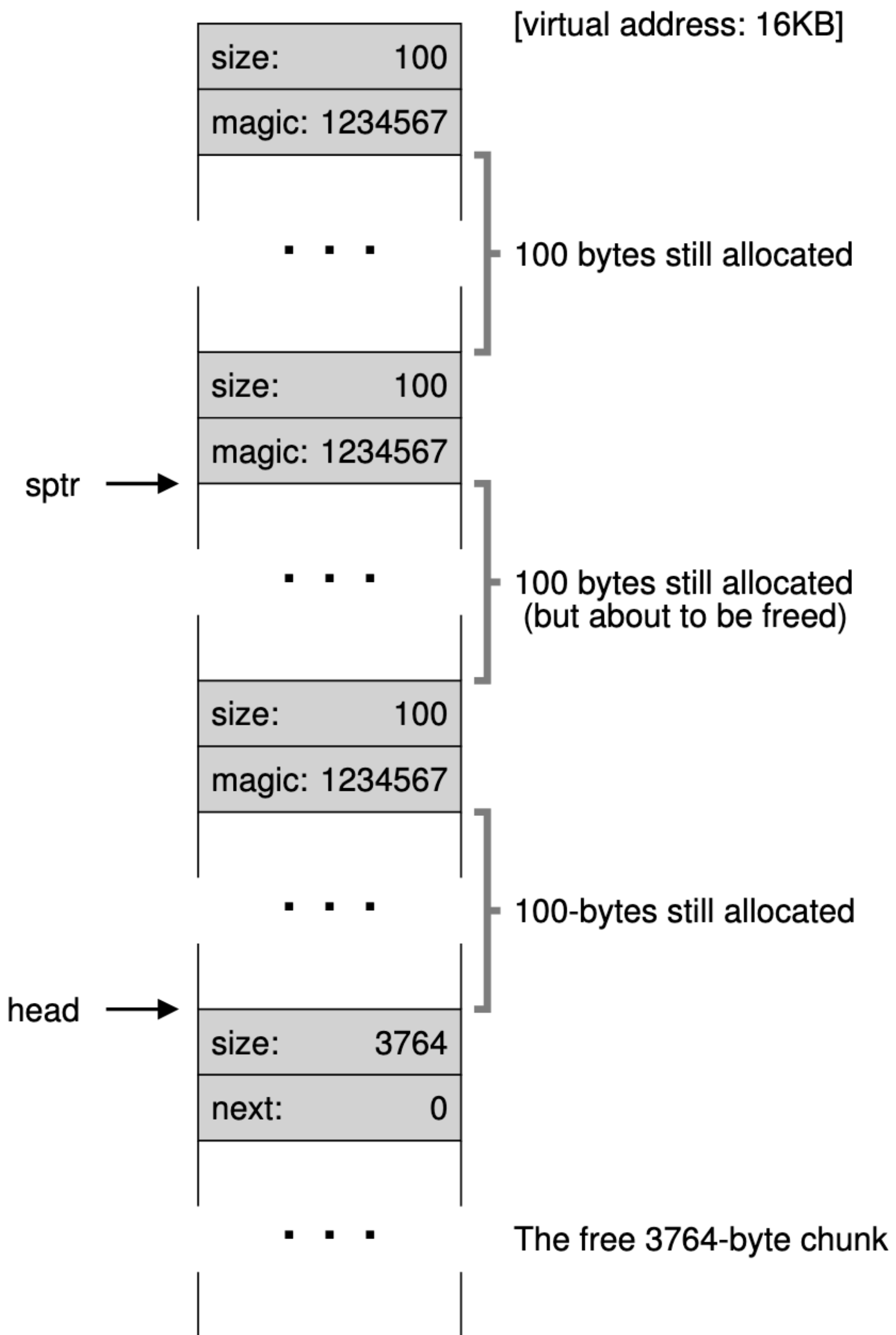


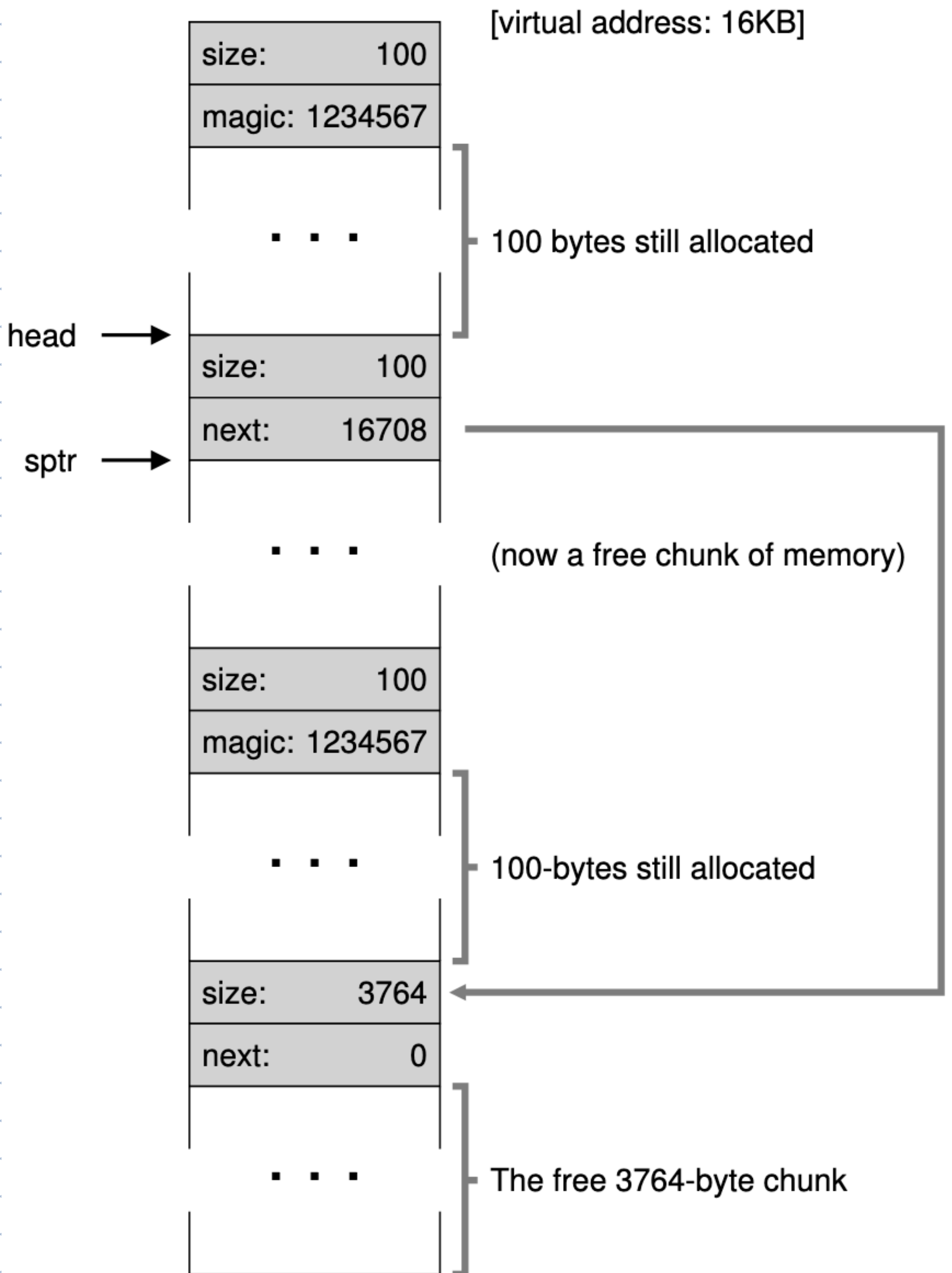
Figure 17.5: Free Space With Three Chunks Allocated

The first 324 bytes are allocated on the heap with one node (chunk) free.

When `free(16500)` is called,

$16,384 + 108 + 8$ size of header = 16,500
 base address / starting address of heap / 1st allocated chunk size / start address of chunk to be freed.

the library immediately figures out the size of free region & adds it to the list, the heap looks like:



Finally, assume the last two allocated chunks are freed, the heap will now be chopped up, appearing fragmented

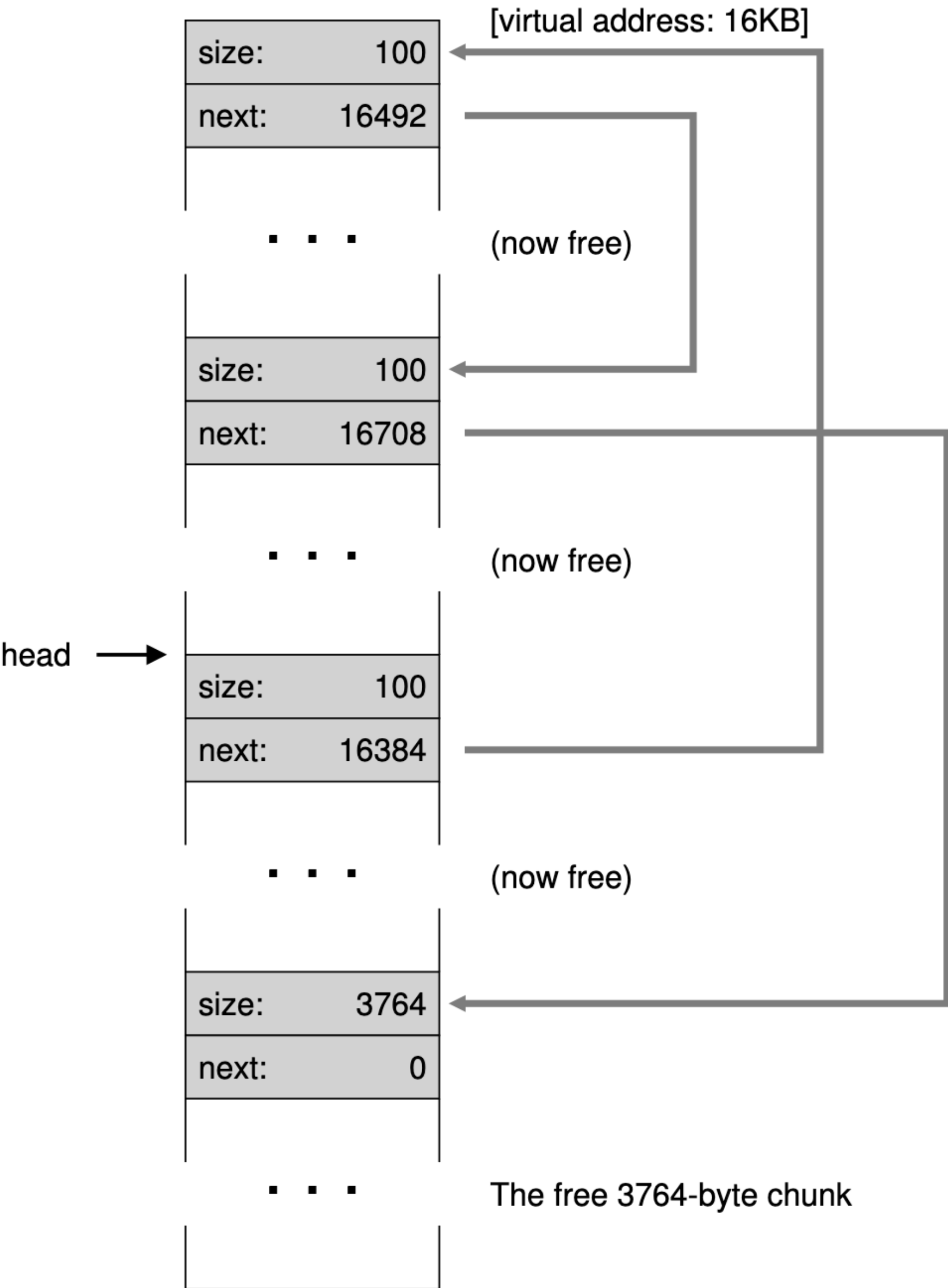


Figure 17.7: A Non-Coalesced Free List

So, go through the list & merge neighbouring chunks.

- Growing the heap

- ↳ Most allocators start with a small heap & then request more from the OS.

Basic Strategies (Policies)

- Best fit:

- ↳ Search through the free list & find chunks of free memory that are as big or bigger than the requested size.

- Then, return the smallest one among them.

- ↳ A con is that you need search the entire list (slow...agh!).

- Worst fit:

- ↳ Instead of smallest, return the largest chunk.

- ↳ Worst fit tries to leave large free chunks behind instead of small ones in best fit.

- Cons:

- ↳ Once again, you need to search the entire list.

- ↳ Leads to excessive fragmentation.

- ↳ Every time you choose largest free space & thus leave some part of it free.

- First Fit:

- ↳ Finds the first block that is big enough & returns the requested amount to user.

- Pros:

- No need to search entire list.

Cons:

Pollutes the start of the list with small chunks.

↳ Ordering the list by address helps with coalescing.

• Next fit:

↳ Keeps an extra pointer to the location within the list where one was looking last.

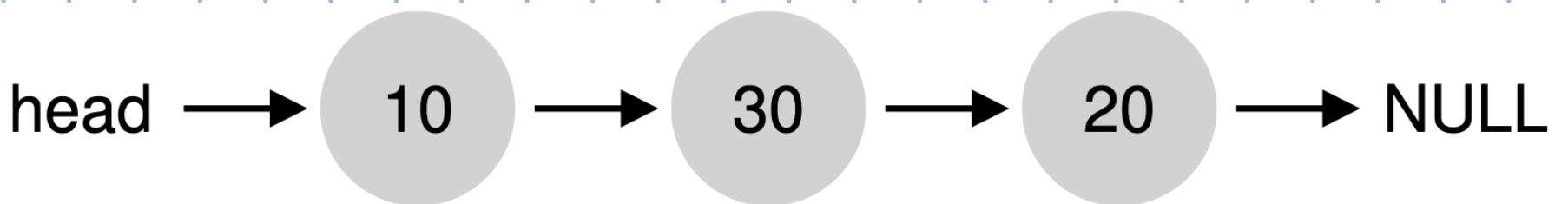
↳ Look from last allocated region

Pros:

↳ Avoids polluting the start & spreads more uniformly.

↳ No need to search entire list.

Example,



Request for 15 bytes:

Best fit:



Worst fit, first fit, Next fit:



Other Approaches:

↳ If a particular process has one (or few) popular-sized request that it makes, keep a separate list just to manage chunks of that size.

All other requests are forwarded to a more general memory allocator.

↳ By having one chunk of memory for one particular size of requests, fragmentation is less of a concern.

Also, allocation & de-allocation is fast no search is required.

Issues with this:

↳ How much memory dedicated for this?

↳ One particular approach:

When kernel boots up, it allocates a no. of object caches for kernel objects that are likely to be requested frequently (PCB, file-system inodes).

They are each segregated free lists of a given size.

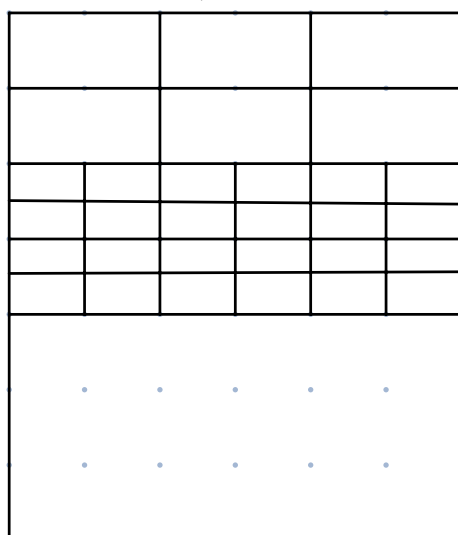
↳ When these lists run low, it requests slabs (fixed size memory chunks) of memory from the general memory allocator.

↳ When reference counts of objects of a given slab go to zero, the general allocator can re-claim them.

Explanation of 5

1. The heap or memory is divided into separate lists or pools for commonly used sizes (e.g., PCB, file descriptors, etc.).
2. Each list contains fixed-size blocks, and an array occupied/unoccupied blocks.
3. When a request for a specific size comes in, the OS finds an empty block in the corresponding list and calculates the address using the base address and index as:
 $\text{base_address_of_fixed_sized_list} + (\text{sizeof}(\text{PCB}) * \text{unoccupied_index})$

Heap



→ PCB →



→ file descriptors

→ Generic

↓
maintain an array indicating occupied/non-occupied slots & simply index into them to

Buddy allocation

↳ Maintain a free-list allocate space like above.

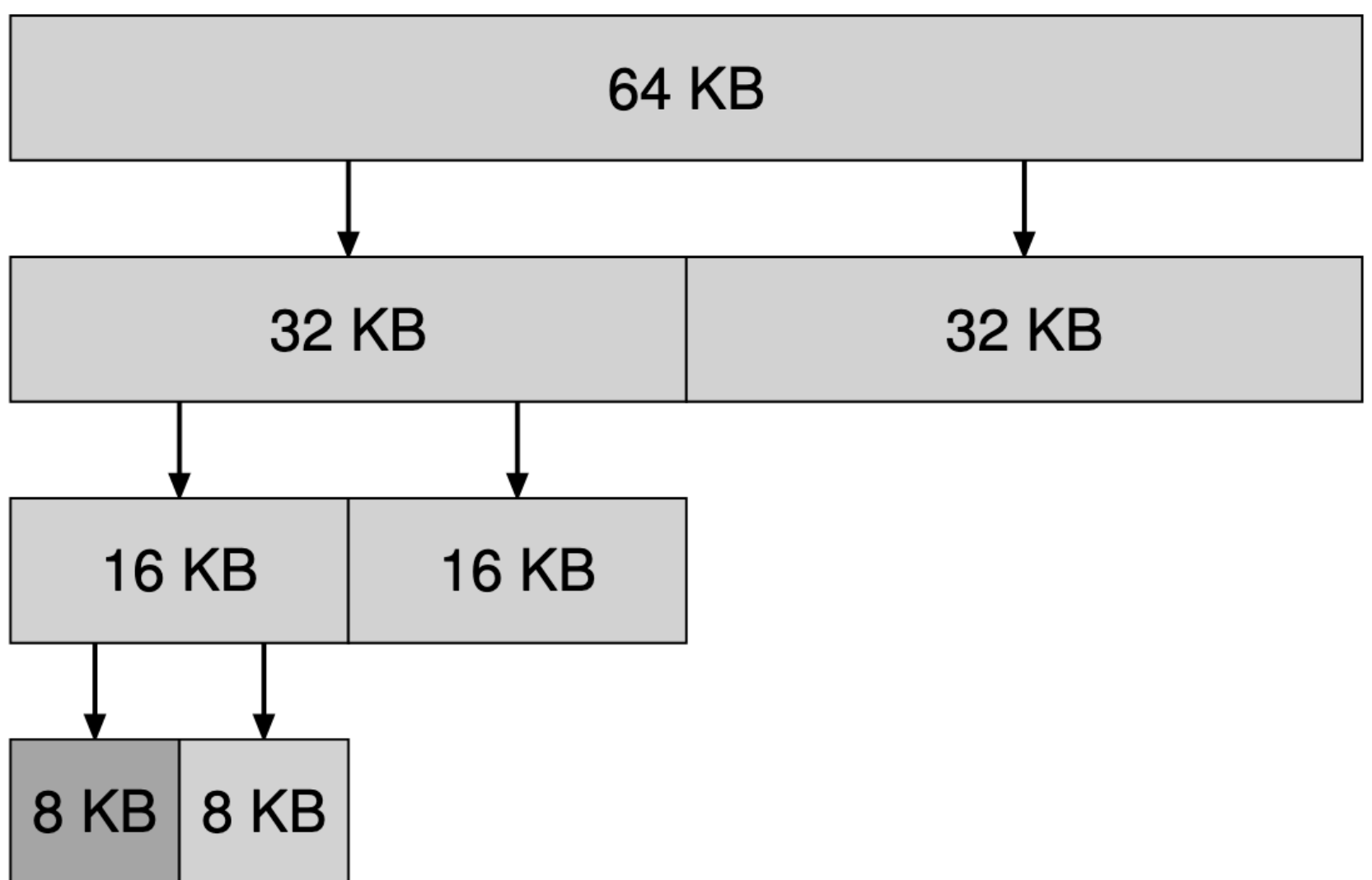
↳ Makes coalescing easier.

↳ Free memory is thought of as one big space of 2^N .

When a request for memory is made, the search for free space divides free space by two until a block that is big enough to accommodate the request is found. Further split would result in a block that is too small.

Example,

A request for 7kB is made to a 64kB free space:



↳ When freeing the 7kB block, the allocator checks if the buddy is free & if so then merges them into one 16kB block.

This continues till it finds a buddy in use or if the entire heap is freed.

The address of buddy differs only by top few bits.